

NR2DT-09: Step 9 — Cutover, Rollback & Decommission

Series: NR2DT — New Relic to Dynatrace Migration Steps | **Notebook:** 9 of 10 |
Created: April 2026 | **Last Updated:** 10/05/2026

Overview

Goal of this step: complete the cutover from NR to DT. Halt NR ingest, archive NR artifacts, and confirm DT is the sole source of truth. Rollback plan stays valid for 30+ days.

Procedural — see **NRLC-08** (rollback manifest mechanics) and **NRLC-09** (toolchain reference) for depth.

Table of Contents

- 1. [Cutover Sequence](#)
- 2. [Rollback Plan](#)
- 3. [Decommission NR](#)
- 4. [Step Exit Criteria](#)

Prerequisites

Requirement	Details
Audience	Migration lead + assigned engineer for this step
Completed	NR2DT-08 — Validate

Requirement	Details
Format	Procedural step — use as a runbook; defer to NRLC for depth
NRLC deep dives	NRLC-08 (rollback), NRLC-09 (toolchain)

1. Cutover Sequence

Cutover is per-component, sequenced.

Order	Component	Cutover Action
1	Dashboards	Disable NR dashboard sharing; communicate DT URLs to consumers
2	Synthetics	Disable NR synthetic monitors (one type at a time)
3	Alerts	Disable the conditions of the remaining NR alert policies (last: critical incident routes)
4	SLOs	Disable NR SLO reports; switch SLA dashboard data source to DT
5	Logs	Stop dual-shipping; NR log ingest disabled per source

New Relic has no on/off switch for a policy itself: *"You can't disable a policy directly. However, you can disable all of the policy's conditions."*

Halt criteria: if any cutover step surfaces a regression, **stop and roll back that component**. Don't proceed to the next.

Sources: [Create, edit, or find an alert policy \(New Relic docs\)](#), read 10/05/2026.

Cutover, Rollback & Decommission Sequence

Per-component cutover · rollback retained 30+ days · NR halted last

Cutover Sequence (per component)



Rollback (manifest-driven)

migrate.py migrate --rollback <manifest>

Deletes EVERY entry in the manifest (no filter)
Run with --dry-run first: lists targets only

Partial rollback = filtered manifest copy
Manifests retained ≥ 30 days post-cutover

Rollback does NOT undo:

- ✗ Data already ingested into Grail (config-only)
- ✗ NR-side changes (NR stays disabled)
- ✗ Manual edits to migrated entities

(only deletes what the manifest records)
For a fresh rebuild after rollback: restart at Step 4.

S9 — Migration Complete

All stakeholders sign off · NR contract action

Cutover is reversible per component until NR ingest is halted; that's the one-way step.

2. Rollback Plan

Every full `migrate` run (no `--dry-run`, no `--import-only`) writes `rollback-manifest.json` into its `--output` directory, listing the entities it created. `--import-only` runs write no manifest. Retain each manifest for **at least 30 days** post-cutover.

Rollback command

```
# Always preview first: lists the targets, deletes nothing
python3 migrate.py migrate --rollback ./run-dashboards/rollback-manifest.json
--dry-run

# Execute (asks for confirmation)
python3 migrate.py migrate --rollback ./run-dashboards/rollback-manifest.json
```

The rollback:

1. Reads the manifest of created entities
2. Lists the targets and asks for confirmation
3. Calls DELETE on each via the DT API
4. Prints each failure and an `ok` / `failed` total

`--rollback` **deletes every entry in the manifest.** There is no per-component filter: `--components` is ignored on this path, so `--rollback <manifest> --components dashboards`

also deletes every alert, SLO and workflow that run created. For a **partial rollback**, write a filtered copy of the manifest (`{"entries": [...]}`) that keeps only the entries whose `entity_type` you want removed, dry-run it, then roll back from the copy:

```
jq '{entries: [.entries[] | select(.entity_type == "anomaly_detector")]}' \
  ./run-alerts/rollback-manifest.json > rollback-detectors-only.json
python3 migrate.py migrate --rollback rollback-detectors-only.json --dry-run
```

What rollback DOES NOT undo:

- Data already ingested into Grail (configuration-only rollback)
- NR-side changes (NR dashboards stay disabled even after rollback)
- Manual edits to migrated entities (only deletes what the manifest records)
- Anything created by an `--import-only` run, or by hand (no manifest entry)

Sources: [migrate.py @ 78cbfce \(tool repo, GitHub\)](#), [migration/state.py @ 78cbfce \(tool repo, GitHub\)](#), read 10/05/2026.

If you need a full rebuild after rollback, treat it as a fresh migration starting from Step 4 (translation is still valid).

3. Decommission NR

Final-stage cleanup. Sequenced for safety.

Order	Action	Reversible?
1	Disable NR alert routing (Workflows / channels)	Yes — re-enable in NR UI
2	Set NR dashboards to private / read-only	Yes
3	Halt NR ingest per source (uninstall / reconfigure agents)	Yes (reinstall)
4	Export NR account snapshot for archive	One-way
5	Cancel NR contract / reduce seat count	Per contract terms

Decommission gate W7 (NR2DT-02 §4): all stakeholders sign off that DT is operational and NR is no longer needed.

4. Step Exit Criteria

S9 — Migration Complete

- ☐ All components cut over to DT
- ☐ Rollback manifests retained per retention policy (≥ 30 days)
- ☐ NR alert routing disabled
- ☐ NR dashboards archived / set to read-only
- ☐ NR ingest halted at all sources
- ☐ NR account snapshot exported and stored
- ☐ All stakeholders signed off on completion

Next step: NR2DT-99 — Best Practice Summary (post-mortem and ongoing best practices).

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source [NewRelic-to-Dynatrace-Migration-Utilities](#), [nrql-engine](#), and [nrql-translator](#) projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official [Dynatrace documentation](#) and [New Relic documentation](#).